



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

گزارش آزمایش هفتم

اعضای گروه

کیارش شجاعی ارانی

سینا فعالیت

پارسا ملکیان

نام درس

آزمایشگاه طراحی سامانه‌های رقمی

نیم‌سال تابستان ۱۴۰۵

نام استاد درس

دکتر اجلالی

۱ مقدمه

UART روشی رایج برای انتقال سریال و ناهمزمان داده است. چون ساعت^۱ مستقلی همراه داده ارسال نمی‌شود، فرستنده و گیرنده باید بر سر نرخ بیت^۲ و قالب قاب^۳ توافق داشته باشند. هدف آزمایش حاضر، طراحی یک فرستنده و گیرنده هفت‌بیتی قابل سنتز، اتصال آن‌ها در مسیر بازگشت داخلی^۴ و اثبات عملکردشان با میزآزمون‌ها است.

اهداف جزئی آزمایش عبارت‌اند از تبدیل داده موازی هفت‌بیتی به دنباله سریال^۵، بازسازی همان داده در گیرنده، تولید و کنترل بیت توازن، کنترل بیت توقف، اعلام وضعیت اشتغال فرستنده، اعلام پایان دریافت و اعتبار قاب، و تحلیل زمانی همه این رخدادها در شکل موج شبیه‌سازی.

۲ قالب قاب و محاسبه توازن

خط سریال در حالت بیکار^۶ مقدار منطقی ۱ دارد. افت خط به صفر آغاز قاب را مشخص می‌کند. پس از بیت شروع^۷، بیت توازن و سپس بیت‌های D0 تا D6 ارسال می‌شوند و در پایان، بیت توقف با مقدار ۱ قرار می‌گیرد. بنابراین، حالت Idle پیش‌شرط آغاز دریافت است و خود قاب شامل ده بیت به ترتیب زیر است:

Idle(1) -> Start(0) -> Parity -> D0 -> D1 -> D2 -> D3 -> D4 -> D5 -> D6
-> Stop(1)

داده هفت‌بیتی به روش LSB-first منتقل می‌شود؛ یعنی D0، کم‌ارزش‌ترین بیت، زودتر از D6، پرارزش‌ترین بیت، روی خط قرار می‌گیرد. بیت توازن با XOR کاهشی همه بیت‌های داده محاسبه می‌شود:

$$\text{Parity} = D_0 \oplus D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_5 \oplus D_6 \quad (1)$$

¹Clock

²Bit Rate

³Frame Format

⁴Internal Loopback

⁵Serial Stream

⁶Idle State

⁷Start Bit

برای داده نمونه 7'b1011001 یا 0x59، بیت‌ها در ترتیب ارسال برابر ۱، ۰، ۱، ۱، ۰، ۰، ۱ هستند. تعداد بیت‌های یک برابر چهار است؛ در نتیجه XOR آن‌ها صفر و بیت توازن برابر ۰ است. قاب کامل این داده پس از خط بیکار چنین توالی‌ای دارد:

Start=0, Parity=0, D0..D6=1,0,0,1,1,0,1, Stop=1

۳ زمان‌بندی و انتخاب پارامتر بیت

در نرخ ۱۱۵۲۰۰ بیت بر ثانیه، زمان هر بیت برابر معکوس این نرخ، یعنی تقریباً ۶۸/۸ میکروثانیه است. برای ساعت ۵۰ مگاهرتز، تعداد چرخه ساعت در هر بیت از رابطه زیر به دست می‌آید:

$$\frac{50\,000\,000}{115\,200} = 434.79 \dots \simeq 434 \quad (2)$$

از این رو مقدار صحیح ۴۳۴ برای BIT_TICKS انتخاب شده است. هر حالت زمانی فرستنده، یک نماد را به مدت ۴۳۴ لبه ساعت نگه می‌دارد و گیرنده نیز با همین مقیاس نمونه‌برداری می‌کند. می‌آزمون اختصاصی قالب قاب از همین مقدار استفاده می‌کند تا الزام واقعی ۱۱۵۲۰۰ بیت بر ثانیه را اثبات کند.

برای شکل موج‌های آموزشی، BIT_TICKS=10 انتخاب شده است. با ساعت ۱۰ نانوثانیه‌ای، هر نماد ۱۰۰ نانوثانیه طول دارد و مرز حالت‌ها، شمارنده و نمونه‌برداری میانی به وضوح قابل مشاهده‌اند. برای آزمون تجمیعی و پیمایش تمام مقادیر، مقدار ۲ زمان اجرا و طول پرونده موج را کاهش می‌دهد، بی‌آنکه ترتیب منطقی قاب تغییر کند. این سه مقدار مکمل یکدیگرند: ۴۳۴ برای دقت نرخ، ۱۰ برای خوانایی موج و ۲ برای پوشش سریع و گسترده.

۴ معماری طرح و اتصال بازگشتی

پرونده UARTSender.v فرستنده و ماشین حالت ارسال را دربر دارد. پرونده UARTReceiver.v گیرنده، همگام‌سازی ورودی، نمونه‌برداری، کنترل توازن و کنترل بیت توقف را پیاده‌سازی می‌کند. پرونده UARTTop.v نمونه‌ای از هر دو پیمانه^۸ می‌سازد و tx فرستنده را مستقیماً به rx گیرنده متصل

^۸Module

می‌کند. پرونده‌ی Tester.v میزآزمون تجمیعی است و ساعت، بازنشانی^۹، کنترل خودکار و پیام‌های نتیجه را تولید می‌کند. میزآزمون‌های مستقل نیز هر رفتار مهم را جداگانه و با شکل موجی خوانا بررسی می‌کنند.

اتصال سطح بالا در خطوط ۱۸ تا ۳۴ پرونده‌ی UARTTop.v آمده است. همان خروجی tx هم به بیرون پیمانۀ ارائه می‌شود و هم ورودی rx نمونه‌گیرنده است:

```
18 UARTSender #(.BIT_TICKS(BIT_TICKS)) sender (  
19     .clk(clk),  
20     .rstN(rstN),  
21     .new_data(new_data),  
22     .send_data(send_data),  
23     .tx(tx),  
24     .busy(busy)  
25 );  
26  
27 UARTReceiver #(.BIT_TICKS(BIT_TICKS)) receiver (  
28     .clk(clk),  
29     .rstN(rstN),  
30     .rx(tx),  
31     .rec_data(rec_data),  
32     .rec_new_data(rec_new_data),  
33     .correct_data(correct_data)  
34 );
```

بنابراین، هر داده‌ی پذیرفته‌شده در فرستنده باید پس از گذشت یک قاب کامل در خروجی گیرنده دیده شود. این بازگشت داخلی، صحت هم‌زمان دو ماشین حالت را بدون نیاز به یک منبع سریال خارجی آزمایش می‌کند.

^۹Reset

۵ طراحی فرستنده

ورودی‌های فرستنده `clk`، `rstN`، `new_data` و `send_data[6:0]` و خروجی‌های آن `tx` و `busy` هستند. بازنشانی فعال-پایین^{۱۰}، حالت را به `IDLE`، شمارنده‌ها و ثبات‌ها را به صفر و خط `tx` را به حالت بیکار منطقی ۱ می‌برد.

ماشین حالت فرستنده پنج حالت `IDLE`، `START`، `PARITY`، `DATA` و `STOP` دارد. خطوط ۴۵ تا ۴۸ ارتباط حالت با دو خروجی اصلی را نشان می‌دهند:

```
45 assign busy = state != IDLE;
46 assign tx = state == START ? 1'b0 :
47         state == PARITY ? parity_latch :
48         state == DATA ? data_latch[bit_index] : 1'b1;
```

سیگنال `busy` از نامساوی بودن حالت جاری با `IDLE` ساخته می‌شود؛ بنابراین، از ورود به `START` تا پایان `STOP` فعال می‌ماند. در `START` مقدار صفر، در `PARITY` مقدار `parity_latch` و در `DATA` بیت انتخاب شده با `bit_index` روی `tx` قرار می‌گیرد. در `STOP` و `IDLE` خروجی ۱ است.

خطوط ۵۹ تا ۱۰۸، قفل کردن ورودی و حرکت اصلی ماشین حالت را نشان می‌دهند:

¹⁰Active-Low

```

59 IDLE: begin
60     tick_count <= {TICK_WIDTH{1'b0}};
61     bit_index  <= 3'd0;
62     if (new_data) begin
63         data_latch  <= send_data;
64         parity_latch <= ^send_data;
65         state       <= START;
66     end
67 end
68
69 START: begin
70     if (tick_count == LAST_TICK) begin
71         tick_count <= {TICK_WIDTH{1'b0}};
72         state <= PARITY;
73     end else begin
74         tick_count <= tick_count + 1'b1;
75     end
76 end
77

```

ادامه ماشین، بیت توازن و سپس هفت بیت داده را مدیریت می‌کند:

```

78  PARITY: begin
79      if (tick_count == LAST_TICK) begin
80          tick_count <= {TICK_WIDTH{1'b0}};
81          bit_index <= 3'd0;
82          state <= DATA;
83      end else begin
84          tick_count <= tick_count + 1'b1;
85      end
86  end
87
88  DATA: begin
89      if (tick_count == LAST_TICK) begin
90          tick_count <= {TICK_WIDTH{1'b0}};
91          if (bit_index == 3'd6) begin
92              state <= STOP;
93          end else begin
94              bit_index <= bit_index + 1'b1;
95          end
96      end else begin
97          tick_count <= tick_count + 1'b1;
98      end
99  end
100

```

قطعه پایانی، بیت توقف را نگه می‌دارد و فرستنده را به حالت بیکار بازمی‌گرداند:

```

101 STOP: begin
102     if (tick_count == LAST_TICK) begin
103         tick_count <= {TICK_WIDTH{1'b0}};
104         state <= IDLE;
105     end else begin
106         tick_count <= tick_count + 1'b1;
107     end
108 end

```

در IDLE، تنها هنگام فعال بودن new_data، ورودی در data_latch و توازن آن در parity_latch ذخیره می‌شود. این قفل کردن^{۱۱} باعث می‌شود تغییر بعدی send_data قاب در حال ارسال را عوض نکند. منطق پذیرش نیز فقط در شاخه IDLE قرار دارد؛ در نتیجه، درخواست جدید در زمان busy بدون اثر است. شمارنده tick_count مدت هر نماد را اندازه می‌گیرد و bit_index در حالت داده از صفر تا شش افزایش می‌یابد تا ترتیب LSB-first مستقیماً از data_latch[bit_index] حاصل شود.

۶ طراحی گیرنده

ورودی‌های گیرنده clk، rstN و rx و خروجی‌های آن [6:0] rec_data، rec_new_data و correct_data هستند. چون rx نسبت به clk ناهمزمان است، ابتدا از دو فلیپ‌فلاپ پیاپی عبور می‌کند. خطوط ۴۹ تا ۵۸ این همگام‌ساز را نشان می‌دهند:

^{۱۱}Latching


```

49 // Two flip-flops isolate the FSM from an asynchronous serial input.
50 always @(posedge clk or negedge rstN) begin
51     if (!rstN) begin
52         rx_meta <= 1'b1;
53         rx_sync <= 1'b1;
54     end else begin
55         rx_meta <= rx;
56         rx_sync <= rx_meta;
57     end
58 end

```

rx_meta نخستین نمونه و rx_sync نمونه همگام‌شده‌ای است که ماشین حالت مصرف می‌کند. این ساختار احتمال رسیدن متاستابیلیتی^{۱۲} به منطق کنترلی را کاهش می‌دهد و تأخیر دو لبه ساعتی قابل پیش‌بینی ایجاد می‌کند.

گیرنده نیز حالت‌های IDLE، START، PARITY، DATA و STOP دارد. خطوط ۷۱ تا ۱۲۹ اعلان‌های پالسی، تأیید میانی شروع، نمونه‌برداری و کنترل نهایی قاب را پیاده‌سازی می‌کنند:

¹²Metastability

```

71 // Completion and validity are aligned one-clock notifications.
72 rec_new_data <= 1'b0;
73 correct_data <= 1'b0;
74
75 case (state)
76     IDLE: begin
77         tick_count <= {TICK_WIDTH{1'b0}};
78         bit_index <= 3'd0;
79         if (!rx_sync)
80             state <= START;
81     end
82
83     START: begin
84         if (tick_count == HALF_TICK) begin
85             tick_count <= {TICK_WIDTH{1'b0}};
86             state <= rx_sync ? IDLE : PARITY;
87         end else begin
88             tick_count <= tick_count + 1'b1;
89         end
90     end
91

```

در ادامه، بیت توازن ذخیره و هفت بیت داده به ترتیب نمونه برداری می شوند:

```

92  PARITY: begin
93      if (tick_count == LAST_TICK) begin
94          tick_count <= {TICK_WIDTH{1'b0}};
95          bit_index <= 3'd0;
96          parity_latch <= rx_sync;
97          state <= DATA;
98      end else begin
99          tick_count <= tick_count + 1'b1;
100      end
101  end
102
103  DATA: begin
104      if (tick_count == LAST_TICK) begin
105          tick_count <= {TICK_WIDTH{1'b0}};
106          data_latch[bit_index] <= rx_sync;
107          if (bit_index == 3'd6) begin
108              state <= STOP;
109          end else begin
110              bit_index <= bit_index + 1'b1;
111          end
112      end else begin
113          tick_count <= tick_count + 1'b1;
114      end
115  end
116

```

قطعه پایانی، بیت توقف و برابری توازن را بررسی و خروجی‌های پالسی را تولید می‌کند:

```

117 STOP: begin
118     if (tick_count == LAST_TICK) begin
119         tick_count <= {TICK_WIDTH{1'b0}};
120         rec_new_data <= 1'b1;
121         if (rx_sync && ((~data_latch) == parity_latch)) begin
122             rec_data <= data_latch;
123             correct_data <= 1'b1;
124         end
125         state <= IDLE;
126     end else begin
127         tick_count <= tick_count + 1'b1;
128     end
129 end

```

در IDLE، صفرشدن rx_sync شروع احتمالی قاب است. ماشین در START تا نیمه بیت صبر می‌کند؛ اگر خط در HALF_TICK دوباره ۱ شده باشد، رخداد یک شروع کاذب تلقی می‌شود و ماشین به IDLE بازمی‌گردد. در غیر این صورت، بیت توازن در parity_latch و هفت نمونه بعدی از D0 تا D6 در data_latch ذخیره می‌شوند.

در پایان STOP، rec_new_data برای دقیقاً یک چرخه ساعت فعال می‌شود و پایان پردازش قاب را اعلام می‌کند. شرط اعتبار هم‌زمان شامل rx_sync=1 برای بیت توقف و برابری parity_latch با XOR کاهشی data_latch است. فقط در صورت برقرار بودن هر دو شرط، rec_data با داده تازه به‌روز و correct_data نیز برای همان یک چرخه ساعت فعال می‌شود. در قاب نامعتبر، اعلان پایان همچنان ایجاد می‌شود، ولی correct_data صفر می‌ماند و rec_data آخرین داده معتبر را حفظ می‌کند.

۷ روش میزآزمون و پوشش آزمون‌ها

ساعت اصلی در میزآزمون‌ها دوره ۱۰ نانوثانیه دارد. در آغاز، rstN صفر می‌شود و سپس مدار در یک لبه کنترل‌شده از بازنشانی خارج می‌گردد. درخواست ارسال با قرار دادن داده روی send_data و فعال‌کردن new_data برای یک چرخه ساعت ساخته می‌شود. کنترل‌گرهای خودکار با انتظار

برای busy و rec_new_data، مقدار rec_data، اعتبار correct_data و پهنای پالس^{۱۳} را بررسی می‌کنند. برای قاب‌های خراب، یک محرک خام^{۱۴} rx بیت‌های قاب را مستقیماً تولید می‌کند تا توازن یا بیت توقف عمداً تغییر داده شود.

سیگنال‌های اصلی قرارگرفته در پنجره Wave عبارت‌اند از clk، rstN، new_data، send_data، tx، busy، rec_data، rec_new_data و correct_data. در آزمون‌های اختصاصی، سیگنال‌های داخلی مانند state، tick_count، bit_index، data_latch، parity_latch، rx_meta، rx_sync و rx_sync نیز برای تحلیل دقیق‌تر افزوده شده‌اند.

جدول ۱ محرک و نتیجه مورد انتظار همه سناریوهای الزامی و تکمیلی را خلاصه می‌کند.
جدول ۱: ماتریس پوشش میزآزمون‌های UART

ردیف	سناریو و محرک	نتیجه مورد انتظار
۱	دریافت صحیح 0x59	rec_data=0x59 و پالس هم‌زمان پایان و اعتبار
۲	قالب دقیق با BIT_Ticks=434	ده بیت قاب، هرکدام به مدت ۴۳۴ چرخه
۳	0x2D با توازن معکوس ۱	پایان قاب، اعتبار صفر و حفظ داده معتبر قبلی
۴	0x2D با Stop=0	رد قاب حتی با توازن صحیح
۵	0x59 و سپس 0x2D	دو دریافت معتبر و مرتب در نخستین فرصت قانونی
۶	درخواست 0x2D هنگام ارسال 0x59	قاب جاری بدون تغییر و نبود دریافت دوم
۷	بازنشانی با send_data=0x7F	خط بیکار، busy=0 و خروجی‌های پاک
۸	قاب معتبر و قاب با توازن خراب	پالس یک‌چرخه‌ای پایان برای هر دو و اعتبار فقط برای قاب سالم
۹	پیمایش 0x00 تا 0x7F	تطابق و اعتبار برای هر ۱۲۸ مقدار
۱۰	پالس شروع ۱۰ نانوثانیه‌ای و سپس 0x59	رد شروع کاذب و بازیابی کامل

میزآزمون tb_uart_frame_format.v مقدار BIT_Ticks را در خط ۵ برابر ۴۳۴ و داده آزمون را در خط ۶ برابر 7'b1011001 قرار می‌دهد. تابع کنترل خطوط ۲۸ تا ۴۰، هر نماد را در تمام ۴۳۴

¹³Pulse Width

¹⁴Raw Driver

چرخه می‌سنجد:

```
5 localparam integer BIT_TICKS = 434;
6 localparam [6:0] TEST_DATA = 7'b1011001;
28 task check_symbol;
29     input expected_tx;
30     integer cycle;
31     begin
32         for (cycle = 0; cycle < BIT_TICKS; cycle = cycle + 1) begin
33             if (tx !== expected_tx || busy !== 1'b1)
34                 $fatal(1,
35                     "FAIL: symbol=%b cycle=%0d tx=%b busy=%b",
36                     expected_tx, cycle, tx, busy);
37             @(negedge clk);
38         end
39     end
40 endtask
```

پس از اعمال بازنشانی و پالس یک چرخه‌ای `new_data`، خطوط ۵۶ تا ۷۱ بیت شروع، توازن، هفت بیت داده و بیت توقف را به همان ترتیب کنترل می‌کنند و بازگشت نهایی به حالت بیکار را می‌سنجند:

```

56  @(negedge clk);
57  send_data = TEST_DATA;
58  new_data = 1'b1;
59  @(negedge clk);
60  new_data = 1'b0;
61
62  check_symbol(1'b0);
63  check_symbol(^TEST_DATA);
64  for (bit_index = 0; bit_index < 7; bit_index = bit_index + 1)
65      check_symbol(TEST_DATA[bit_index]);
66  check_symbol(1'b1);
67
68  if (tx !== 1'b1 || busy !== 1'b0)
69      $fatal(1, "FAIL: sender did not return to idle");
70
71  $display("PASS: sender frame format and 434-cycle timing");

```

پایش پالس‌های خروجی در خطوط ۶۲ تا ۹۳ پرونده Tester.v انجام می‌شود. علاوه بر ثبت هر تکمیل، این بخش خطا می‌دهد اگر rec_new_data بیش از یک چرخه بماند یا correct_data خارج از چرخه تکمیل فعال شود:

```

62 // Record every completion and enforce the one-cycle output contract.
63 always @(negedge clk or negedge rstN) begin
64     if (!rstN) begin
65         top_completion_count = 0;
66         raw_completion_count = 0;
67         previous_top_new = 1'b0;
68         previous_raw_new = 1'b0;
69     end else begin
70         if (rec_new_data && previous_top_new)
71             $fatal(1, "FAIL: loopback rec_new_data exceeded one cycle");
72         if (raw_rec_new_data && previous_raw_new)
73             $fatal(1, "FAIL: raw rec_new_data exceeded one cycle");
74         if (correct_data && !rec_new_data)
75             $fatal(1, "FAIL: loopback correct_data is not completion-aligned");
76         if (raw_correct_data && !raw_rec_new_data)
77             $fatal(1, "FAIL: raw correct_data is not completion-aligned");
78     end

```

بخش دوم پایش، داده و اعتبار هر تکمیل را ثبت و وضعیت چرخه پیشین را نگهداری می‌کند:


```

79         if (rec_new_data) begin
80             top_data_history[top_completion_count] = rec_data;
81             top_correct_history[top_completion_count] = correct_data;
82             top_completion_count = top_completion_count + 1;
83         end
84         if (raw_rec_new_data) begin
85             raw_data_history[raw_completion_count] = raw_rec_data;
86             raw_correct_history[raw_completion_count] = raw_correct_data;
87             raw_completion_count = raw_completion_count + 1;
88         end
89
90         previous_top_new = rec_new_data;
91         previous_raw_new = raw_rec_new_data;
92     end
93 end

```

سناریوهای خطای توازن، خطای توقف و شروع کاذب در خطوط ۲۷۹ تا ۳۰۸ همان پرونده
به صورت خودبررسی نوشته شده‌اند:

```

279 $display("TEST: required parity error");
280 base_count = raw_completion_count;
281 drive_raw_frame(SECOND_DATA, ~(^SECOND_DATA), 1'b1);
282 wait_raw_count(base_count + 1);
283 if (raw_correct_history[base_count] !== 1'b0 ||
284     raw_data_history[base_count] !== FIRST_DATA)
285     $fatal(1, "FAIL: parity error semantics");
286
287 $display("TEST: required stop-bit error");
288 base_count = raw_completion_count;
289 drive_raw_frame(SECOND_DATA, ^SECOND_DATA, 1'b0);
290 wait_raw_count(base_count + 1);
291 if (raw_correct_history[base_count] !== 1'b0 ||
292     raw_data_history[base_count] !== FIRST_DATA)
293     $fatal(1, "FAIL: stop-bit error semantics");

```

آزمون شروع کاذب و بازیابی پس از آن در قطعه بعدی آمده است:

```

294
295 $display("TEST: false-start rejection and recovery");
296 base_count = raw_completion_count;
297 @(negedge clk);
298 raw_rx = 1'b0;
299 @(negedge clk);
300 raw_rx = 1'b1;
301 repeat (4 * BIT_TICKS) @(negedge clk);
302 if (raw_completion_count != base_count)
303     $fatal(1, "FAIL: false start completed a frame");
304 drive_raw_frame(SECOND_DATA, ^SECOND_DATA, 1'b1);
305 wait_raw_count(base_count + 1);
306 if (raw_correct_history[base_count] !== 1'b1 ||
307     raw_data_history[base_count] !== SECOND_DATA)
308     $fatal(1, "FAIL: recovery after false start");

```

در آزمون مستقل همهٔ مقادیر، حلقهٔ خطوط ۴۵ تا ۶۶ هر کد هفت‌بیتی را ارسال می‌کند، پایان دریافت را با مهلت محدود انتظار می‌کشد، داده و اعتبار را می‌سنجد و یک چرخه‌ای بودن دو پرچم را نیز کنترل می‌کند:

```

45 for (value = 0; value < 128; value = value + 1) begin
46     @(negedge clk);
47     send_data = value[6:0];
48     new_data = 1'b1;
49     @(negedge clk);
50     new_data = 1'b0;
51
52     cycles = 0;
53     while (rec_new_data != 1'b1 && cycles < TIMEOUT_CYCLES) begin
54         @(negedge clk);
55         cycles = cycles + 1;
56     end
57     if (rec_new_data != 1'b1)
58         $fatal(1, "FAIL: value %0d timed out", value);
59     if (correct_data != 1'b1 || rec_data != value[6:0])
60         $fatal(1, "FAIL: sent=%0d received=%0d correct=%b",
61             value, rec_data, correct_data);
62
63     @(negedge clk);
64     if (rec_new_data != 1'b0 || correct_data != 1'b0)
65         $fatal(1, "FAIL: output pulse width for value %0d", value);
66 end

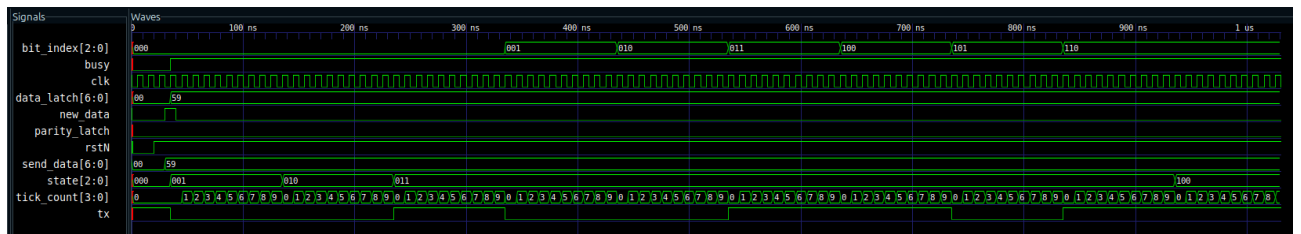
```

۸ تحلیل شکل موج‌های شبیه‌سازی

شکل‌های این بخش خروجی اجرای میزآزمون‌ها با Icarus Verilog و مشاهده در GTKWave هستند. هر تحلیل، محرک دقیق، سیگنال‌های قابل مشاهده و نتیجه کنترل خودکار را در کنار هم بیان می‌کند.

۱-۸ ارسال و دریافت معتبر

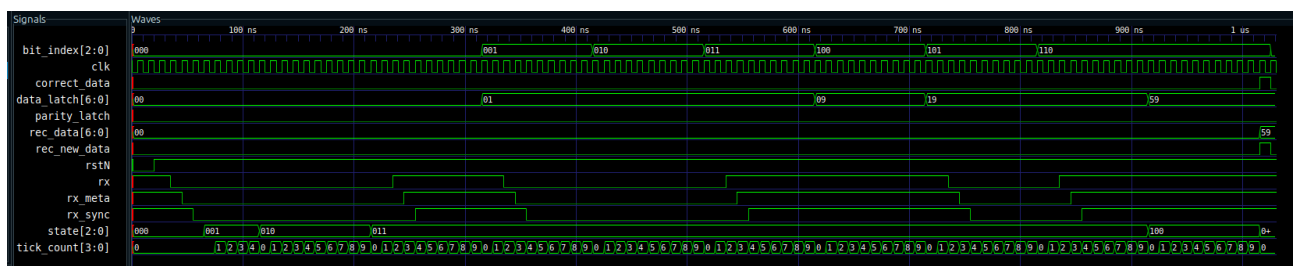
رفتار فرستنده برای داده نمونه در شکل ۱ دیده می‌شود.



شکل ۱: شکل موج فرستنده برای ارسال معتبر داده 7'b1011001

در شکل ۱، TEST_DATA برابر 7'b1011001 یا 0x59 است. برای new_data یک چرخه فعال می‌شود؛ در همان پذیرش، data_latch مقدار 0x59 و parity_latch مقدار ۰ را نگه می‌دارند و busy فعال می‌شود. tx ابتدا Start=0، سپس Parity=0، بعد دنباله D0 تا D6 برابر ۱، ۰، ۰، ۱، ۰، ۱ و سرانجام Stop=1 را تولید می‌کند. هر بخش با BIT_TICKS=10 دقیقاً ده چرخه ساعت پایدار است. تغییرات bit_index مرز بیت‌های داده را مشخص می‌کند و بازگشت state به IDLE هم‌زمان با غیرفعال شدن busy است. در نتیجه، قفل داده، محاسبه توازن، ترتیب LSB-first و زمان‌بندی فرستنده صحیح‌اند.

بازسازی همان قاب در گیرنده در شکل ۲ نمایش داده شده است.

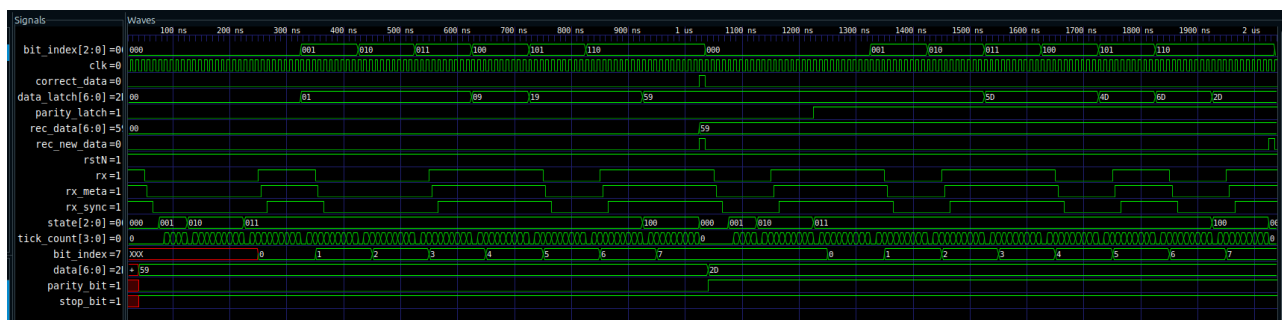


شکل ۲: شکل موج گیرنده برای دریافت معتبر داده 7'b1011001

در شکل ۲، rx سیگنال واقعی خط و rx_meta و rx_sync به ترتیب نسخه‌های یک و دو مرحله همگام‌شده آن هستند؛ بنابراین، هم‌تراز نبودن دقیق سه موج عمودی و ناشی از دو فلیپ‌فلاپ همگام‌ساز است، نه تصادفی بودن نرخ بیت. ماشین پس از مشاهده صفر وارد START می‌شود، در میانه بیت آن را تأیید می‌کند، parity_latch=0 را می‌گیرد و سپس data_latch را از D0 تا D6 تکمیل می‌کند. در پایان، Stop=1 است و توازن محاسبه‌شده صفر با بیت ذخیره‌شده برابر است؛ از این رو rec_data به 0x59 تغییر می‌کند و rec_new_data و correct_data هم‌زمان برای یک چرخه بالا می‌روند. این موج بازسازی صحیح و اعتبارسنجی نهایی قاب را اثبات می‌کند.

۲-۸ خطای توازن

واکنش گیرنده به بیت توازن خراب در شکل ۳ دیده می شود.

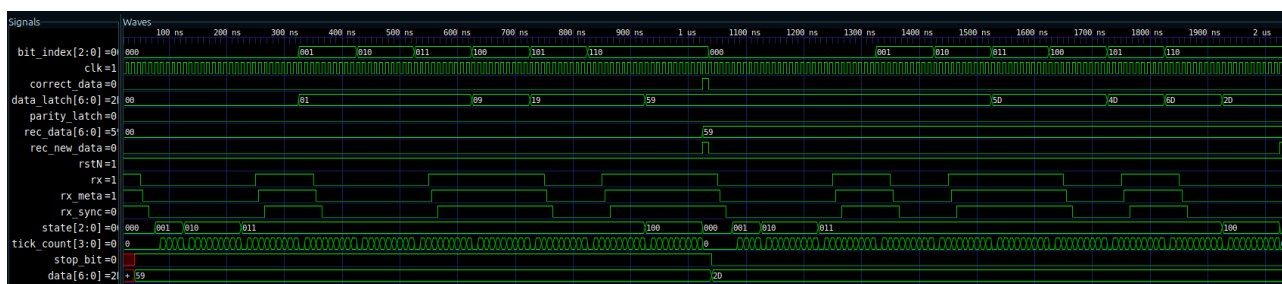


شکل ۳: شکل موج دریافت 0x2D با بیت توازن معکوس

در بخش نخست شکل ۳، قاب 0x59 معتبر دریافت می شود تا مقدار شناخته شده 0x59 داشته باشد. قاب دوم داده 7'b0101101 یا 0x2D را حمل می کند. این داده نیز چهار بیت یک دارد و توازن صحیح آن صفر است، اما محرک خام عمداً بیت توازن دریافتی را ۱ می کند. پس از دریافت هفت بیت و توقف صحیح، گیرنده پایان قاب را با پالس یک چرخه ای rec_new_data اعلام می کند؛ ولی چون parity_latch=1 با XOR داده، یعنی ۰، برابر نیست، correct_data فعال نمی شود و rec_data نیز 0x59 باقی می ماند. پس پالس پایان با پالس اعتبار یکسان نیست و خطای توازن از ورود داده خراب به خروجی جلوگیری می کند.

۳-۸ خطای بیت توقف

اثر صفر کردن عمدی بیت توقف در شکل ۴ نشان داده شده است.



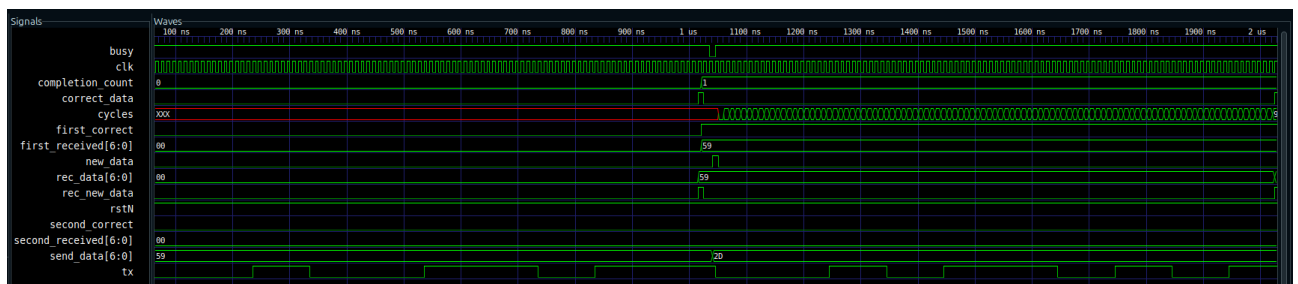
شکل ۴: شکل موج دریافت 0x2D با بیت توقف صفر

در شکل ۴ نیز قاب معتبر 0x59 ابتدا خروجی را مقداردهی می کند. قاب خام بعدی حاوی 0x2D و توازن صحیح صفر است، اما Stop عمداً صفر نگه داشته می شود. چون D6 داده 0x2D نیز صفر

است، D6 و بیت توقف خراب در موج به شکل یک بازه صفر طولانی و بدون لبه میان آن‌ها دیده می‌شوند. در پایان بازه، rec_new_data یک چرخه فعال می‌شود، اما شرط rx_sync=1 برقرار نیست؛ در نتیجه correct_data صفر و rec_data برابر 0x59 باقی می‌ماند. حتی با داده و توازن صحیح، Stop=0 قاب را نامعتبر می‌کند.

۴-۸ قاب‌های متوالی

ترتیب دریافت دو قاب الزامی در شکل ۵ قابل مشاهده است.

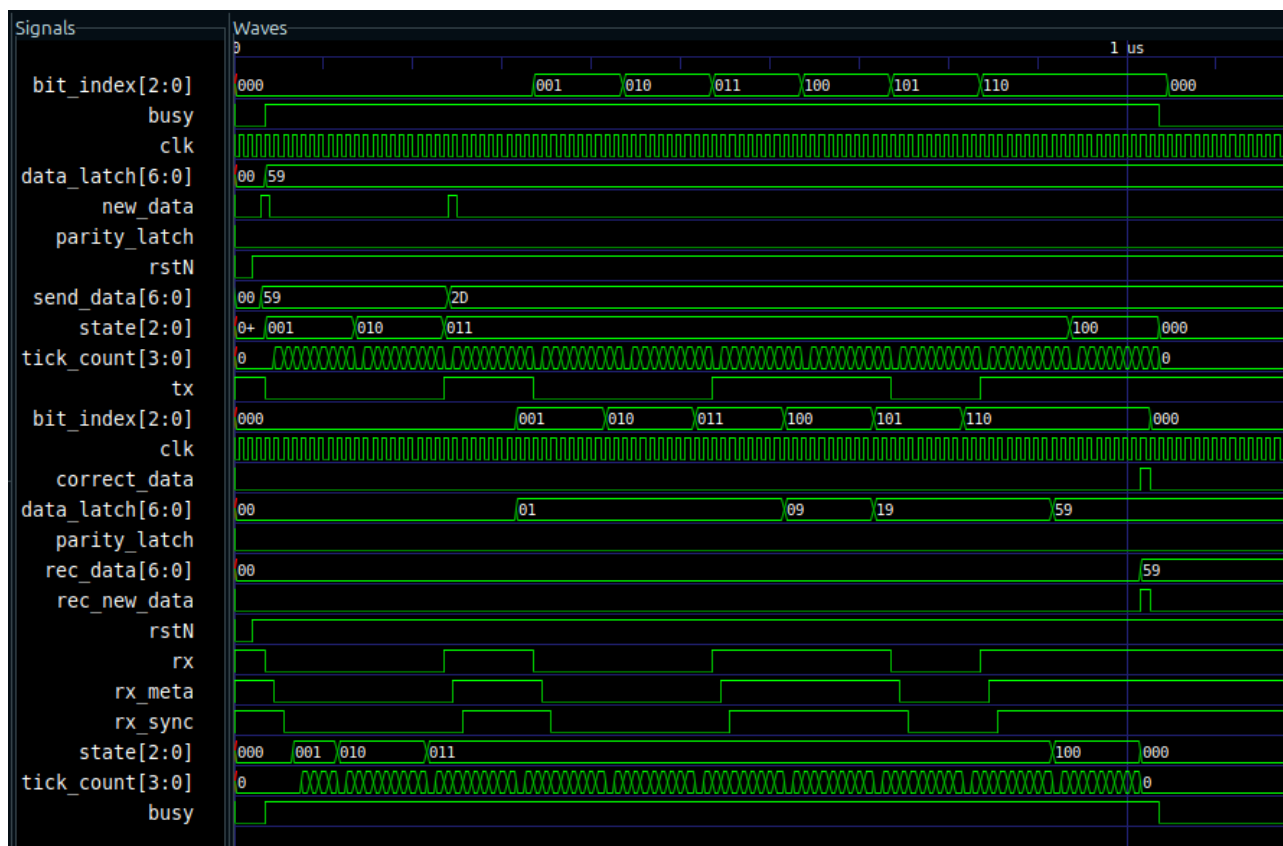


شکل ۵: شکل موج ارسال متوالی 0x59 و 0x2D

در شکل ۵ ابتدا 0x59 و سپس 0x2D ارسال می‌شود. درخواست دوم در نخستین فرصت قانونی پس از پایین آمدن busy اعمال شده است. میان دو قاب یک بازه کوتاه IDLE و busy=0 دیده می‌شود که اجازه پذیرش داده دوم را می‌دهد؛ سپس busy دوباره برای قاب بعدی فعال می‌شود. گیرنده دو پالس rec_new_data و correct_data تولید می‌کند. نمونه ثبت شده نخست 0x59 و نمونه دوم 0x2D است. ترتیب دو مقدار و وجود دقیقاً دو تکمیل معتبر نشان می‌دهد قاب‌ها بدون جابه‌جایی یا ازدست رفتن داده پردازش می‌شوند.

۵-۸ نادیده گرفتن درخواست هنگام اشتغال

آزمون محافظت از قاب فعال در شکل ۶ آمده است.

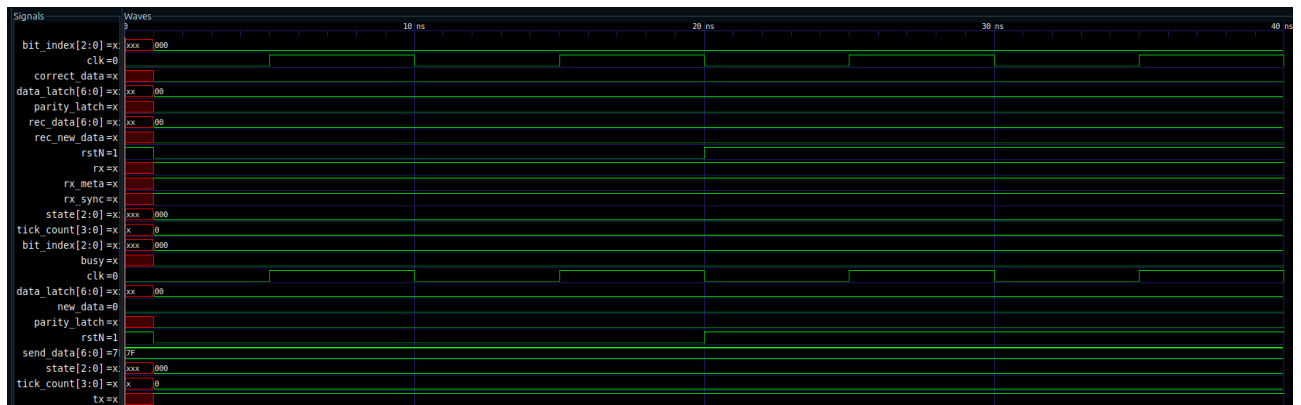


شکل ۶: شکل موج درخواست 0x2D هنگام ارسال فعال 0x59

در شکل ۶ ارسال 0x59 آغاز می‌شود و data_latch همین مقدار را نگه می‌دارد. در میانه busy، send_data به 0x2D تغییر می‌کند و new_data دوباره برای یک چرخه فعال می‌شود. چون منطق پذیرش فقط در IDLE قرار دارد، ثبات داده تغییر نمی‌کند و tx ادامه همان قاب 0x59 را نمایش می‌دهد. در پایان تنها یک پالس دریافت معتبر با rec_data=0x59 دیده می‌شود و پس از زمان کافی هیچ پالس دومی ایجاد نمی‌گردد. پس درخواست هنگام اشتغال نه قاب جاری را مخدوش می‌کند و نه یک قاب ناخواسته در صف می‌سازد.

۸-۶ بازنشانی و حالت بیکار

مقادیر تعریف‌شده مدار در بازنشانی و بیکاری در شکل ۷ دیده می‌شوند.

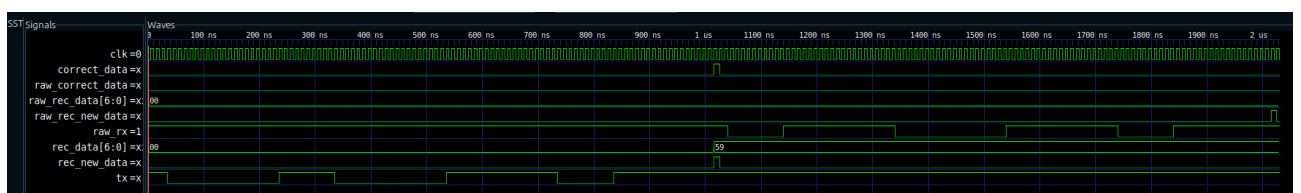


شکل ۷: شکل موج بازنشانی فعال-پایین و پایداری حالت بیکار

در شکل ۷، send_data از ابتدا 0x7F است، اما new_data صفر نگه داشته می‌شود. rstN بازنشانی فعال-پایین را تقریباً از ۱ تا ۲۰ نانوثانیه اعمال می‌کند. مقادیر قرمز X در لحظه نخست، وضعیت طبیعی پیش از اثرکردن بازنشانی هستند. با فعال‌شدن بازنشانی، حالت، شمارنده‌ها و ثبات‌ها صفر می‌شوند، rx_meta و rx_sync به ۱ می‌رسند، tx=1، busy=0، rec_data=0 و خروجی‌های پالسی صفر می‌شوند. پس از آزادشدن rstN تا پایان بازه ۴۰ نانوثانیه، مدار در IDLE پایدار می‌ماند. ثابت‌بودن send_data به‌تنهایی انتقالی آغاز نمی‌کند و نیاز به new_data تأیید می‌شود.

۷-۸ پالس‌های خروجی گیرنده

پهنای اعلان پایان و اعتبار در شکل ۸ مقایسه شده است.



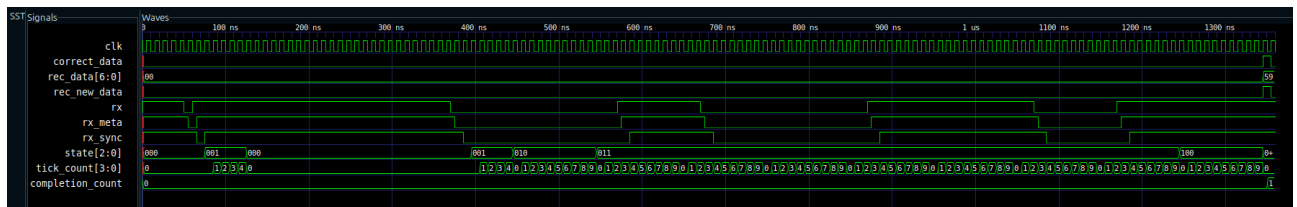
شکل ۸: مقایسه پالس‌های خروجی برای قاب معتبر و قاب دارای خطای توازن

در شکل ۸ نخست یک انتقال بازگشتی معتبر 0x59 انجام می‌شود. نزدیک ۱ میکروثانیه، rec_data=0x59 و rec_new_data و correct_data دقیقاً برای یک دوره clk فعال می‌شوند و در لبه بعدی پایین می‌آیند. سپس ورودی خام همان داده 0x59 را با بیت توازن معکوس ۱ اعمال می‌کند. نزدیک ۳/۲ میکروثانیه، raw_rec_new_data برای یک دوره ساعت فعال می‌شود، اما raw_correct_data همواره صفر و raw_rec_data برابر صفر باقی می‌ماند. این مقایسه نشان می‌دهد اعلان پایان برای هر قاب کامل ایجاد می‌شود، در حالی که اعلان اعتبار فقط به قاب سالم تعلق دارد و هر دو خروجی

ماهیت پالسی یک چرخه‌ای دارند.

۸-۸ شروع کاذب و بازیابی

رد پالس کوتاه خط و دریافت صحیح بعدی در شکل ۹ نمایش داده شده است.

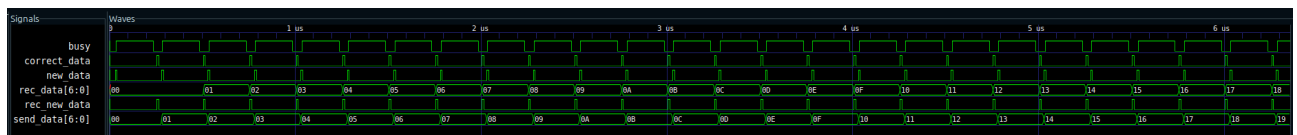


شکل ۹: شکل موج رد شروع کاذب ۱۰ نانوثانیه‌ای و بازیابی با 0x59

با BIT_TICKS=10 و دوره ساعت ۱۰ نانوثانیه، نیمه زمان بیت ۵۰ نانوثانیه است. در شکل ۹، rx در حدود ۵۰ نانوثانیه فقط برای یک چرخه، یعنی ۱۰ نانوثانیه، صفر می‌شود. پس از عبور پالس از rx_sync و rx_meta، ماشین ممکن است موقتاً وارد START شود، اما هنگام تأیید میانی خط دوباره ۱ است و ماشین مستقیماً به IDLE بازمی‌گردد. تا این مرحله نه اعلان پایان و نه اعتبار فعال نمی‌شوند و شمار تکمیل‌ها صفر می‌ماند. قاب معتبر بعدی با داده 0x59 به‌طور کامل دریافت می‌شود، rec_data=0x59 و هر دو پرچم برای یک چرخه فعال می‌شوند و completion_count به ۱ می‌رسد. پس گیرنده هم شروع کوتاه کاذب را رد می‌کند و هم بدون بازنشانی بازیابی می‌شود.

۸-۹ پیمایش تمام مقادیر هفت‌بیتی

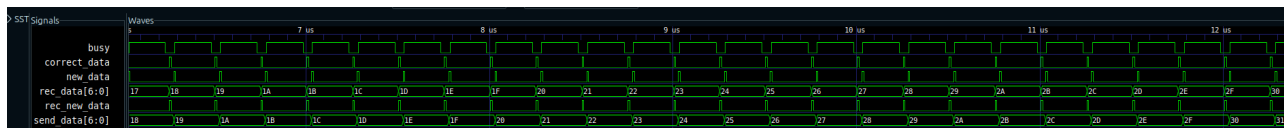
آغاز پیمایش در شکل ۱۰ دیده می‌شود.



شکل ۱۰: بخش نخست شکل موج پیمایش تمام مقادیر هفت‌بیتی

شکل ۱۰ آغاز پیمایش را نشان می‌دهد. در پنجره قابل مشاهده، send_data تقریباً از 0x00 تا 0x19 پیش می‌رود و rec_data با یک قاب تأخیر، مقادیر تقریباً 0x00 تا 0x18 را دنبال می‌کند. برای هر مقدار، یک پالس rec_new_data و یک پالس correct_data دیده می‌شود.

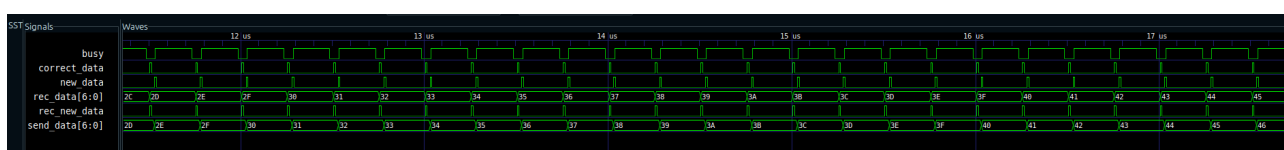
ادامه نخست دنباله در شکل ۱۱ آمده است.



شکل ۱۱: بخش دوم شکل موج پیمایش تمام مقادیر هفت بیتی

شکل ۱۱ ادامه پیمایش را در حدود $\text{send_data}=0x18..0x31$ و $\text{rec_data}=0x17..0x30$ نمایش می دهد. افزایش مرتب داده و جفت شدن هر دریافت با پالس اعتبار نشان می دهد مرز قاب ها در دنباله طولانی حفظ شده است.

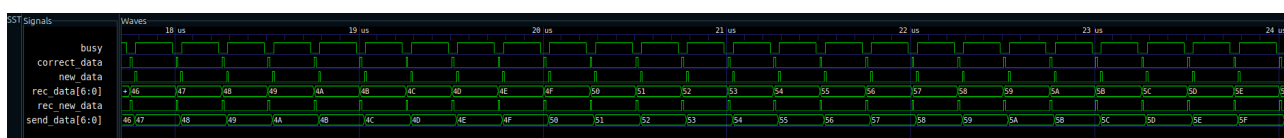
بخش میانی دنباله در شکل ۱۲ دیده می شود.



شکل ۱۲: بخش سوم شکل موج پیمایش تمام مقادیر هفت بیتی

شکل ۱۲ بخش میانی آزمون را در حدود $\text{send_data}=0x2D..0x46$ و $\text{rec_data}=0x2C..0x45$ نشان می دهد. مقادیر دارای الگوهای متفاوت صفر و یک بدون خطای توازن یا ترتیب بیت بازسازی می شوند.

آخرین پنجره موجود در شکل ۱۳ آمده است.



شکل ۱۳: بخش چهارم شکل موج پیمایش تمام مقادیر هفت بیتی

شکل ۱۳ ادامه آزمون را در حدود $\text{send_data}=0x46..0x60$ و $\text{rec_data}=0x46..0x5F$ نشان می دهد. این چهار تصویر فقط پنجره های پیوسته ای از اجرای طولانی هستند؛ پوشش کامل از خود حلقه میزآزمون و نتیجه خودبررسی آن حاصل می شود. حلقه مقدار value را از ۰ تا ۱۲۷ تغییر می دهد، برای هر مقدار انتقال را آغاز می کند، منتظر rec_new_data می ماند و برابری rec_data با مقدار ارسالی و فعال بودن correct_data را کنترل می کند. بنابراین، همه ۱۲۸ کد هفت بیتی از 0x00 تا 0x7F آزمایش شده اند، هرچند تمام آن ها هم زمان در یک تصویر جا نمی گیرند.

۹ خروجی متنی شبیه‌ساز

میزآزمون تجمیعی با دستورهای $\$display$ نام هر مرحله و نتیجه نهایی را چاپ می‌کند. خروجی کامل اجرای موفق به صورت زیر است:

```
PASS: reset and idle outputs
TEST: exact valid frame and symbol timing
TEST: request asserted while busy
TEST: required consecutive frames
TEST: exhaustive seven-bit loopback
TEST: valid raw receiver frame
TEST: required parity error
TEST: required stop-bit error
TEST: false-start rejection and recovery
PASS: complete DSDL7 UART regression
```

این خروجی، Transcript متنی مورد نیاز را فراهم می‌کند. رسیدن اجرا به خط پایانی PASS به این معناست که هیچ‌یک از کنترل‌های خطای داخلی میزآزمون فعال نشده‌اند. نتایج موج و خروجی متنی یکدیگر را تکمیل می‌کنند: شکل موج ترتیب زمانی را نشان می‌دهد و کنترل خودکار، مقدار و پهنای پالس را در لبه‌های ساعت بررسی می‌کند.

۱۰ سنتز و بررسی ساختار در Quartus

خلاصه نتیجه و منابع سنتز در شکل ۱۴ آمده است.

Compilation Report - q7		UARTTop.v	UARTSender.v	UARTReceiver.v
Table of Contents		Analysis & Synthesis Summary		
- Flow Summary - Flow Settings - Flow Non-Default Global Settings - Flow Elapsed Time - Flow OS Summary - Flow Log - Analysis & Synthesis - Summary - Settings - Parallel Compilation - Source Files Read - Resource Usage Summary - Resource Utilization by Entity - State Machines - Optimization Results - Parameter Settings by Entity Instance - Elapsed Time Per Partition - Messages - Fitter - Flow Messages - Flow Suppressed Messages - Assembler - TimeQuest Timing Analyzer		Analysis & Synthesis Status: Successful - Sun Aug 23 19:22:45 2026 Quartus II 64-Bit Version: 13.0.1 Build 232 06/12/2013 SP 1 53 Web Edition Revision Name: q7 Top-level Entity Name: UARTTop Family: Cyclone IV GX Total logic elements: 106 Total combinational functions: 91 Dedicated logic registers: 61 Total registers: 61 Total pins: 21 Total virtual pins: 0 Total memory bits: 0 Embedded Multiplier 9-bit elements: 0 Total G/B Receiver Channel PCS: 0 Total G/B Receiver Channel PMA: 0 Total G/B Transmitter Channel PCS: 0 Total G/B Transmitter Channel PMA: 0 Total PLLs: 0		

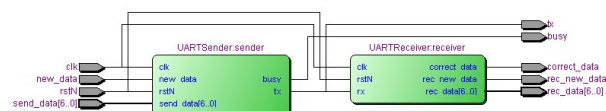
شکل ۱۴: خلاصه موفق Analysis & Synthesis و مصرف منابع پیمانۀ UARTTop

شکل ۱۴ صفحه Analysis & Synthesis Summary را پس از اجرای موفق سنتز نشان می‌دهد. وضعیت Analysis & Synthesis برابر Successful، نام بازبینی q7، پیمانۀ سطح بالا UARTTop و خانواده قطعه Cyclone IV GX است. برای طرح ۱۰۶ عنصر منطقی^{۱۵}، ۹۱ تابع ترکیبی^{۱۶}، ۶۱ ثبات اختصاصی، ۶۱ ثبات در مجموع و ۲۱ پایه گزارش شده است. تعداد بیت‌های حافظه، ضرب‌کننده‌های ۹ بیتی، PLLها و منابع فرستنده‌گیرنده پرسرعت همگی صفر هستند. بنابراین، این تصویر هم موفقیت سنتز و هم هزینه واقعی منابع طرح را مستند می‌کند.

ساختار سلسله‌مراتبی سطح بالا در شکل ۱۵ دیده می‌شود.

¹⁵Logic Elements

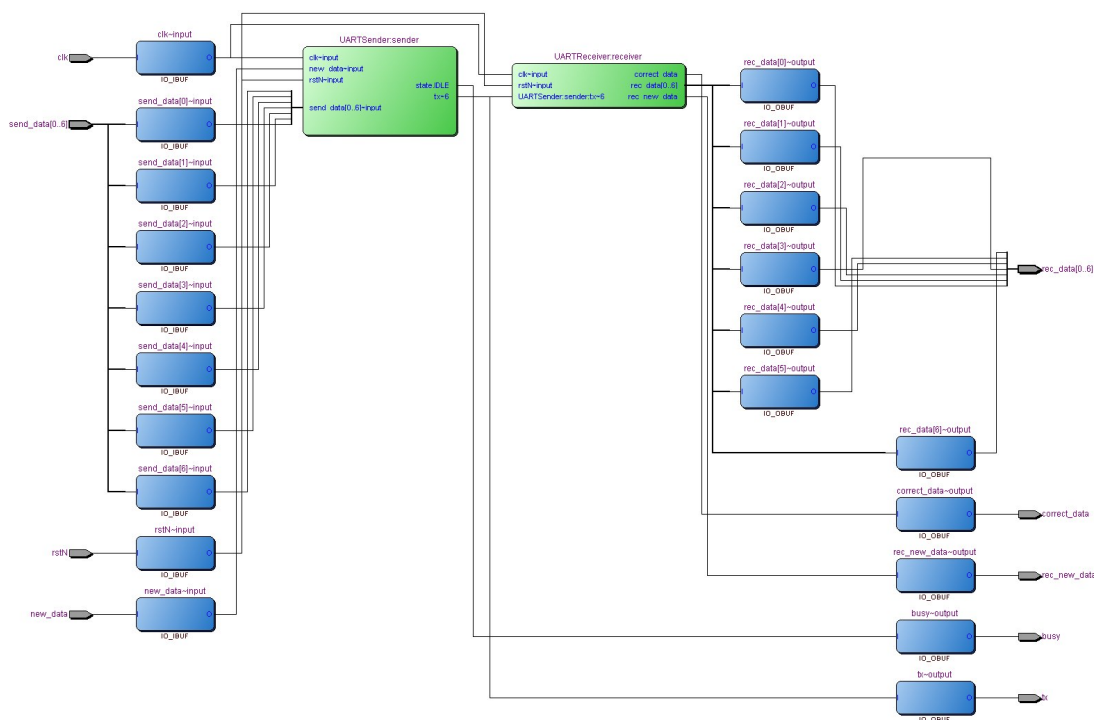
¹⁶Combinational Functions



شکل ۱۵: نمای سلسله‌مراتبی RTL از فرستنده، گیرنده و مسیر بازگشتی داخلی

شکل ۱۵ دو بلوک UARTSender و UARTReceiver را به‌وضوح جدا نشان می‌دهد. ورودی‌های clk و rstN به هر دو بلوک می‌رسند و new_data و send_data[6:0] ورودی‌های فرستنده‌اند. خروجی tx فرستنده هم به درگاه خروجی tx و هم مستقیماً به ورودی rx گیرنده متصل شده است. خروجی‌های busy، rec_data[6:0]، rec_new_data و correct_data نیز از بلوک مربوط به خود به درگاه‌های سطح بالا می‌رسند. بنابراین، نما اتصال بازگشتی داخلی مورد نیاز آزمایش را تأیید می‌کند.

نگاشت تفصیلی شبکه سنتز شده در شکل ۱۶ ارائه شده است.



شکل ۱۶: نمای تفصیلی شبکه سنتز شده و بافرهای ورودی و خروجی پیمانۀ UARTTop

شکل ۱۶ ده ورودی شامل `clk`، هفت بیت `send_data`، `rstN` و `new_data` را نشان می‌دهد که با بافرهای ورودی از نوع `IO_IBUF` وارد ساختار می‌شوند. در سمت خروجی نیز هفت بیت `rec_data` به همراه `rec_new_data`، `correct_data` و `tx` از یازده بافر خروجی از نوع `IO_OBUF` عبور می‌کنند؛ در نتیجه مجموع ۲۱ پایه با خلاصه سنتز یکسان است. تفکیک بیت‌های دو گذرگاه هفت‌بیتی و حفظ دو بلوک فرستنده و گیرنده، نگاشت درست رابط سطح بالا را نشان می‌دهد.

نحوه تشخیص ماشین حالت گیرنده در شکل ۱۷ ثبت شده است.

Table of Contents		State Machine - [UARTTop]UARTReceiver:receiver[state]						
Flow Summary		Encoding Type: One-Hot						
Flow Settings								
Flow Non-Default: Global Settings								
Flow Elapsed Time								
Flow OS Summary								
Flow Log								
Analysis & Synthesis								
Summary								
Settings								
Parallel Compilation								
Source Files Read								
Resource Usage Summary								
Resource Utilization by Entity								
State Machines								
State Machine - [UARTTop]UARTReceiver:receiver[state]								
State Machine - [UARTTop]UARTSender:sender[state]								
Optimization Results								
Register Statistics								
Multiplexer Statistics								
Parameter Settings by Entity Instance								
Elapsed Time Per Partition								
Messages								
Fitter								
Flow Messages								
Flow Suppressed Messages								
Assembler								
TimeQuest Timing Analyzer								

	Name	state.DATA	state.PARITY	state.START	state.IDLE	state.STOP
1	state.IDLE	0	0	0	0	0
2	state.START	0	0	1	1	0
3	state.PARITY	0	1	0	1	0
4	state.DATA	1	0	0	1	0
5	state.STOP	0	0	0	1	1

شکل ۱۷: تشخیص پنج حالت گیرنده و کدگذاری یک‌داغ آن‌ها در گزارش State Machines

شکل ۱۷ گزارش State Machines را برای ثبات state نمونه UARTReceiver نشان می‌دهد. Quartus پنج حالت IDLE، START، PARITY، DATA و STOP را تشخیص داده و Encoding Type را One-Hot انتخاب کرده است؛ در هر ردیف تنها بیت متناظر با همان حالت مقدار ۱ دارد. هرچند حالت‌ها در کد Verilog با مقادیر سه‌بیتی تعریف شده‌اند، ابزار سنتز آن‌ها را برای پیاده‌سازی به کدگذاری یک‌داغ^{۱۷} تبدیل کرده است، بی‌آنکه ترتیب یا رفتار ماشین حالت تغییر کند. در درخت سمت چپ، ماشین حالت فرستنده نیز شناسایی شده است، اما جدول باز شده مشخصاً به گیرنده تعلق دارد.

۱۱ تحلیل نهایی نتایج

برای داده 0x59، خروجی سریال دقیقاً مطابق ترتیب Start، Parity، D0 تا D6 و Stop ساخته شد و گیرنده همان 0x59 را همراه پالس اعتبار بازسازی کرد. آزمون قالب با BIT_TICKS=434 زمان واقعی هر بیت را کنترل می‌کند، در حالی که شکل موج‌های BIT_TICKS=10 همان منطق را با وضوح دیداری بیشتر نشان می‌دهند. بنابراین، کاهش پارامتر در شبیه‌سازی تغییر پروتکل نیست، بلکه تنها

¹⁷One-Hot Encoding

فشرده‌سازی مقیاس زمانی است.

آزمون‌های توازن و توقف نشان دادند شرط اعتبار از دو کنترل مستقل تشکیل شده است. در هر دو خطا، تکمیل قاب اعلام شد، اما correct_data صفر ماند و rec_data به مقدار نامعتبر تغییر نکرد. این رفتار امکان تمایز میان «قاب پردازش شده» و «داده قابل استفاده» را فراهم می‌کند. آزمون busy نیز ثابت کرد قفل ورودی، قاب فعال را در برابر تغییرات send_data و درخواست اضافی محافظت می‌کند. آزمون شروع کاذب نقش نمونه‌برداری میانی را نشان داد و پیمایش ۱۲۸ مقدار، عملکرد را فراتر از یک بردار نمونه پوشش داد.

۱۲ نتیجه‌گیری

فرستنده و گیرنده UART هفت‌بیتی مطابق قالب ویژه آزمایش طراحی شده‌اند. ماشین حالت فرستنده قاب را با توازن XOR و ترتیب LSB-first تولید می‌کند و busy محدوده اشتغال را دقیقاً مشخص می‌سازد. گیرنده ورودی ناهمزمان را همگام، شروع را در میانه بیت تأیید، داده را بازسازی و توازن و توقف را کنترل می‌کند. اتصال بازگشتی سطح بالا و مجموعه آزمون‌های خودکار، دریافت صحیح، رد قاب خراب، حفظ داده معتبر، رفتار بازنشانی، پالس‌های یک‌چرخه‌ای، قاب‌های متوالی، رد درخواست هنگام اشتغال، بازیابی پس از شروع کاذب و تمام فضای داده هفت‌بیتی را پوشش می‌دهند. در نتیجه، رفتار منطقی و زمانی طراحی با الزامات آزمایش هفتم سازگار است.